

Hospital Management System

Tanya Mistry, Krishang Shah

Master of Science in Computer Science
Northeastern University

Abstract

This report presents the design and implementation of a Hospital Management System developed in Python using Tkinter for the interface and SQLite as the backend database. The main goal of the project is to apply the concepts of database design, application development, and data analysis introduced in the course Introduction to Data Management. The system functions as an admin-focused tool for managing essential hospital operations including patient records, appointments, room allocations, treatments, and billing. It also includes analytical queries that offer insights into hospital activity. This report outlines the motivation, system goals, database modeling choices, architectural structure, and potential future enhancements.

1 Introduction

Hospitals need accurate, well-structured information to manage patient care, clinical workflows, and administrative routines. Many smaller facilities still rely on manual processes or outdated tools, leading to scattered records and limited ability to analyze operations. The aim of this project is to design a compact yet meaningful system that demonstrates how proper database structures and user-friendly interfaces can support smoother hospital administration.

Our goal was to build a functional prototype that focuses on correctness, normalization, data validation, and clear relational structure. Although the interface is intentionally simple, the system captures key hospital processes and supports analytical queries that reflect real-world reporting needs.

2 Project Objectives

The main objectives of the system are:

- Design a clean, normalized relational database that represents core hospital entities.
- Implement a graphical application supporting full CRUD operations.
- Enforce correctness through validation rules and foreign key constraints.
- Provide meaningful analytical SQL queries that reveal patient and financial trends.
- Seed the database with realistic sample data for demonstration and testing.

This system is meant purely for academic purposes—it does not include multi-user authentication or production-level security. However, the design leaves room for future growth.

3 System Overview

The Hospital Management System is divided into five main modules:

- Patient Registry
- Appointments
- Room Assignments
- Treatments
- Billing and Invoices

Each module supports adding, viewing, updating, searching, and deleting records. Tkinter is used to create a consistent full-screen layout with a clean, hospital-friendly design.

SQLite is used as the backend because it is lightweight and ideal for prototyping. The system initializes its schema and seeds data on startup so users always begin with a consistent dataset.

4 Architecture

The system follows a straightforward layered architecture.

4.1 User Interface Layer

The graphical interface is built using Python's Tkinter library. Each module is structured as its own window and class but follows a common layout and theme. Forms, tables, and buttons allow smooth navigation for an administrative user.

4.2 Application Logic Layer

This layer includes validation, formatting, and coordination between the interface and the database. All CRUD logic is handled here, along with filtering, searching, and refreshing table views after operations.

4.3 Database Layer

SQLite provides persistent storage. The database schema is created automatically during startup. All tables include primary keys, foreign keys, and check constraints to preserve data integrity. Indexes on frequently queried fields improve responsiveness.

5 Database Design

A relational database model was chosen because it provides clarity and enforces well-structured relationships between entities such as patients, appointments, and invoices. The design aims to avoid redundancy while keeping the schema intuitive for users.

5.1 Tables

The main tables in the system include:

- **Patient:** demographic and contact details.
- **Appointment:** scheduling information, doctor details, and visit reasons.
- **Room Assignment:** inpatient stays and room charges.
- **Treatment:** procedures performed and associated costs.

- **Invoice:** patient billing and financial summaries.

All related tables reference the patient table via foreign keys. Timestamps allow trend analysis over time.

5.2 Normalization

The database is normalized up to Third Normal Form.

Key points:

- No repeating information across tables.
- Each attribute depends solely on its table's primary key.
- Foreign keys enforce relationships instead of storing redundant text fields.

We intentionally kept doctor name and department as simple text fields for simplicity in this prototype. Future versions can extend this with a dedicated doctor table.

5.3 Entity Relationship Diagram

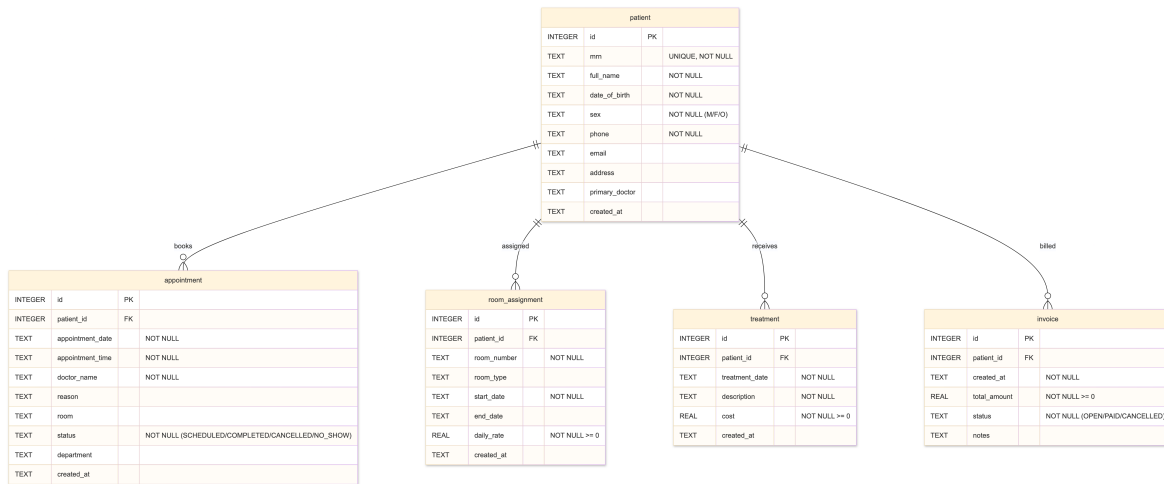


Figure 1: Entity Relationship Diagram for the Hospital Database

6 Implementation

6.1 Technology Stack

The project is implemented in Python, using Tkinter for the graphical interface and SQLite for database storage. The code is organized by functional modules, and initialization scripts automatically create tables and seed data when the program starts.

6.2 Graphical Interface

The interface is designed to be clear and usable, with a full-screen layout and simple navigation. Each module combines an input form with a data table. Users can add, edit, delete, and search records easily.

6.3 Validation

To maintain data correctness, the system runs validation checks before storing or updating records. Examples include:

- Ensuring mandatory fields such as patient name or appointment date are not empty.
- Verifying that numerical fields like costs or rates are non-negative.
- Basic format checks for email and phone inputs.

7 Analytics

The system includes several analytical SQL queries to showcase real-world reporting scenarios:

- Monthly patient visit frequency
- Appointments per doctor per week
- Revenue trends across months
- Ratio of new vs. returning patients
- Billing averages by department

- Appointment cancellation/no-show statistics

These queries demonstrate joins, aggregation, grouping, and subqueries—showing that the database design supports meaningful insights without extensive preprocessing.

8 Testing

Testing was performed in two major areas:

8.1 Usability

We verified that common tasks could be completed without confusion. Testing included adding patients, modifying appointments, validating updated tables, and checking search accuracy.

8.2 Performance

Although the system is small, indexes were added to optimize frequent lookups, especially appointment dates and invoice statuses. This helps maintain smooth performance even as data grows.

9 Future Enhancements

Future improvements to the system could include:

- A full doctor module with authentication and role-based access.
- Migrating from SQLite to MySQL or PostgreSQL.
- A REST API to separate backend logic from the interface.
- Interactive dashboards for real-time analytics.
- Automated appointment reminders via email or SMS.

10 Conclusion

This project demonstrates the key principles of relational database design, GUI programming, and analytical SQL. The Hospital Management System provides a strong foundation with normalized tables, reliable validations, intuitive workflows, and realistic sample data. Overall, it shows how the concepts learned in Introduction to Data Management can be applied to build a fully functional application.